Introduction to Knowledge Engineering

Assignment – CO1

Individual Assignment · Duration: 1 Week · Total Marks: 50

CO1: Apply fundamental concepts of knowledge engineering such as knowledge representation, knowledge acquisition, rule-based reasoning, semantic networks, frames, ontologies and knowledge graphs to model, acquire and manage domain knowledge.

Topics covered: Knowledge Representation – Knowledge Acquisition – Knowledge-Based Systems – Expert Systems – Semantic Networks – Frames – Production Rules – Ontologies – Knowledge Graphs – Knowledge Management Systems.

Instructions: Q1, Q2, Q4, Q5, Q6 and Q7 ask you to build your own examples, so every submission should differ. Make sure every trace, calculation and table follows from your own data. Q3 uses the fixed rule base given in the paper. Diagrams may be hand-drawn or digital.

Q1 – Represent Domain Knowledge (Emergency Triage) — 6 Marks

Problem Context – Hospital Triage. A hospital wants to capture the knowledge of experienced triage nurses to assign each arriving patient a priority of Critical, Urgent or Routine. The nurses use symptoms such as chest pain, blood pressure, body temperature and age.

Your Challenge

1. Identify at least three pieces of knowledge in the triage domain and classify each as declarative, procedural or heuristic knowledge.
2. Represent three of them in predicate logic.
3. Represent the same knowledge as production rules and as a frame describing a patient.
4. Evaluate logic, rules and frames against representational adequacy, inferential adequacy, inferential efficiency and acquisitional efficiency.
5. Identify two difficulties in representing triage knowledge (for example incomplete, uncertain or ambiguous knowledge).

Analysis Inquiry
Using the triage domain, explain the difference between data, information and knowledge.

Evidence Expected: *Classified knowledge items, logic and rule representations, a patient frame, and an evaluation of representation schemes.*

Answer

1.1 Knowledge items and their types

#	Knowledge item	Type
K1	A body temperature above 38 °C is a fever.	Declarative (a fact about the domain)
K2	Assess airway first, then breathing, then circulation.	Procedural (how to carry out a task)
K3	A patient over 50 with chest pain and high blood pressure should be treated as critical.	Heuristic (experience-based rule of thumb)

1.2 Predicate logic

- K1: $\forall x\ (\text{Patient}(x) \wedge \text{Temperature}(x) > 38 \rightarrow \text{Fever}(x))$

- K3: ∀x (Patient(x) ∧ Age(x) > 50 ∧ ChestPain(x) ∧ HighBP(x) → Priority(x, Critical))
- Extra: ∀x (Fever(x) ∧ Age(x) < 5 → Priority(x, Urgent))

1.3 Production rules and a frame

- R1: IF temperature > 38 THEN fever = yes
- R2: IF age > 50 AND chest_pain = yes AND bp = high THEN priority = Critical
- R3: IF fever = yes AND age < 5 THEN priority = Urgent

Frame: Patient	Slot	Value
Class slots	name, age, chest_pain, bp, temperature, priority	chest_pain default = No; bp default = Normal; priority = if-needed procedure (apply R2, R3)
Instance: Ramesh	age = 58, chest_pain = Yes, bp = High	priority = Critical (computed)

1.4 Evaluation of the schemes

Criterion	Predicate logic	Production rules	Frames
Representational adequacy	High – precise, expressive	Medium – good for heuristics	High for structured objects
Inferential adequacy	High – sound inference	Medium – chaining only	Medium – inheritance and defaults
Inferential efficiency	Low – theorem proving is costly	High – simple matching	High – direct slot lookup
Acquisitional efficiency	Low – needs logic skills	High – experts think in rules	Medium – needs a good class model

Rules suit the nurses' heuristic knowledge, frames suit the patient record, and logic is best kept for a precise core of facts.

1.5 Difficulties

- **Uncertain and vague knowledge:** "high" blood pressure and "severe" pain have no sharp threshold, so borderline patients fall between rules.
- **Incomplete knowledge:** an unconscious patient cannot report symptoms, so rules that need them cannot fire; defaults may be wrong.

Analysis inquiry – data, information, knowledge

Data is a raw value such as "150/95". **Information** is that value given meaning: Ramesh's blood pressure is high. **Knowledge** is the general rule that lets the system use it: a patient over 50 with chest pain and high blood pressure needs immediate care. Data and information describe one patient; knowledge applies to every patient and drives action.

Q2 – Acquire Knowledge from an Expert (Crop Disease Advisory) — 6 Marks

Problem Context – Paddy Disease Advisory. A state agriculture department wants an advisory system that helps farmers identify paddy diseases. A plant-pathologist gave the following statement during a first interview:

Expert statement

"When paddy leaves show brown oval spots with grey centres and the weather has been humid for several days, I suspect blast. If the spots are mainly on the lower leaves and the farmer has used a lot of nitrogen fertiliser, I become more certain. But if the whole leaf is turning yellow with no spots, I would think of nutrient deficiency rather than a disease, and I would first ask for a soil test."

Your Challenge

1. Identify the roles in the project (domain expert, knowledge engineer, end user) and at least three sources of knowledge other than the expert.
2. Select three knowledge-elicitation techniques and justify each for this domain.
3. Prepare five interview questions that would follow up the statement above.
4. Convert the statement into at least four production rules and mark where certainty is involved.
5. Explain the knowledge-acquisition bottleneck, identify two difficulties in this project, and state how you would validate the acquired rules.

Analysis Inquiry

Why is tacit knowledge difficult to elicit, and which technique would help most in capturing it?

Evidence Expected: *Roles and sources, justified techniques, interview questions, rules derived from the transcript, and a validation plan.*

Answer

2.1 Roles and sources

- **Domain expert:** the plant-pathologist. **Knowledge engineer:** elicits, structures and encodes the knowledge. **End user:** farmers and extension officers who use the advisory.
- **Other sources:** department bulletins and crop-protection manuals, records of past field cases, research papers, and extension officers' field experience.

2.2 Elicitation techniques

Technique	Why it suits this domain
Structured interview	Efficient for explicit knowledge: symptoms, thresholds, treatments and exceptions.
Observation / protocol analysis (think-aloud)	The expert diagnoses real or photographed leaves while speaking aloud, exposing visual pattern recognition that interviews miss.
Card sorting	The expert groups photos of symptoms into disease categories, revealing the distinctions he or she actually uses.

2.3 Follow-up interview questions

1. How many days of humid weather make you suspect blast, and what temperature range matters?
2. How exactly do blast spots differ from brown spot or sheath blight spots?
3. What counts as "a lot" of nitrogen fertiliser, in kg per acre?
4. Which findings would make you rule blast out completely?
5. When you suspect nutrient deficiency, which nutrients do you check first and what does the soil test confirm?

2.4 Production rules (CF = confidence assigned with the expert)

Rule	IF	THEN	CF
R1	spots = brown_oval_grey_centre AND humid_days = several	disease = blast	0.6
R2	R1 conditions AND spots_on = lower_leaves AND nitrogen = high	disease = blast	0.85
R3	leaf = whole_yellowing AND spots = none	suspect = nutrient_deficiency	0.7
R4	suspect = nutrient_deficiency	action = request_soil_test	1.0

The words “suspect” and “more certain” show uncertainty, so R1–R3 carry certainty factors; R4 is a fixed procedure. The numbers are my assumptions and must be confirmed with the expert.

2.5 Bottleneck, difficulties and validation

- **Knowledge-acquisition bottleneck:** eliciting and encoding expert knowledge is slow and expensive, and is the main barrier to building knowledge-based systems.
- **Difficulties here:** (i) much of the diagnosis is tacit visual judgement; (ii) different experts may disagree on thresholds or the importance of weather and fertiliser.
- **Validation:** run the rules on past cases with known diagnoses, ask a second expert to review them, run a field trial and measure agreement with expert diagnoses.

Analysis inquiry – tacit knowledge
Tacit knowledge is skill gained through experience that the expert cannot easily put into words: the pathologist ‘just sees’ blast. Asking questions yields only what the expert can articulate and may produce idealised rules. **Observation with think-aloud protocol analysis** helps most, because the expert is caught in the act of diagnosing and the engineer can record the cues that are used, then confirm them with card sorting.

Q3 – Reasoning with Production Rules (Loan Pre-screening) — 8 Marks

Problem Context – Bank Loan Pre-screening. A bank encodes its pre-screening knowledge as the following rule base. Rules are tried in the order R1 to R6.

Rule	IF (conditions)	THEN (conclusion)
R1	Salaried AND Income_Stable	Employment_Secure
R2	Credit_Score_Good AND No_Defaults	Good_Credit
R3	Employment_Secure AND Good_Credit	Low_Risk
R4	Low_Risk AND Amount_Within_Limit	Approve_Loan
R5	Self_Employed AND Income_Stable AND Has_Collateral	Employment_Secure
R6	Low_Risk AND Amount_Exceeds_Limit	Refer_To_Manager

Case A facts: Salaried, Income_Stable, Credit_Score_Good, No_Defaults, Amount_Within_Limit.

Case B facts: Self_Employed, Income_Stable, Has_Collateral, Credit_Score_Good, No_Defaults, Amount_Exceeds_Limit.

Your Challenge

1. Apply forward chaining to Case A. Show the rule fired at each step and the facts added to working memory.
2. Apply backward chaining to Case A with the goal Approve_Loan. Show the goals and sub-goals.

3. Apply forward chaining to Case B. State whether Approve_Loan can be derived and what the system concludes.
4. Explain what backward chaining would do for the goal Approve_Loan in Case B, and which question the system would have to ask the user.
5. Describe two conflict-resolution strategies and state which one you used.
6. Explain how contradictory, redundant or circular rules would affect a rule-based system and how they can be detected.

> **Analysis Inquiry**
> When is forward chaining preferable to backward chaining, and when is backward chaining preferable?

Evidence Expected: *Step-by-step traces, goal tree, working-memory contents, and interpretation of the conclusions.*

Answer

Conflict resolution used: rules are scanned in order R1 → R6, the first applicable rule that has not already fired is fired, and the scan restarts (rule order with refraction).

3.1 Case A – forward chaining

Step	Rule fired	Working memory afterwards
0	–	Salaried, Income_Stable, Credit_Score_Good, No_Defaults, Amount_Within_Limit
1	R1	… + Employment_Secure
2	R2	… + Good_Credit
3	R3	… + Low_Risk
4	R4	… + Approve_Loan

R5 and R6 never fire (Self_Employed and Amount_Exceeds_Limit are absent). Conclusion: the loan is approved.

3.2 Case A – backward chaining (goal: Approve_Loan)

- Approve_Loan ← R4: needs Low_Risk and Amount_Within_Limit.
- Low_Risk ← R3: needs Employment_Secure and Good_Credit.
- Employment_Secure ← R1: Salaried (fact ✓) and Income_Stable (fact ✓), so proved. R5 is not tried because R1 succeeded.
- Good_Credit ← R2: Credit_Score_Good ✓ and No_Defaults ✓, so proved.
- Low_Risk proved; Amount_Within_Limit is a fact ✓, so **Approve_Loan is proved**.

3.3 Case B – forward chaining

Step	Rule fired	Reason / fact added
1	R2	Credit_Score_Good, No_Defaults → Good_Credit (R1 fails: Salaried absent)
2	R5	Self_Employed, Income_Stable, Has_Collateral → Employment_Secure
3	R3	Employment_Secure, Good_Credit → Low_Risk
4	R6	Low_Risk, Amount_Exceeds_Limit → Refer_To_Manager

R4 cannot fire because Amount_Within_Limit is not true, so **Approve_Loan is not derived**. The system concludes that the application is low risk but exceeds the limit, so it is referred to a manager.

3.4 Case B – backward chaining

For the goal Approve_Loan, R4 needs Low_Risk and Amount_Within_Limit. Low_Risk is proved through R3, R5 and R2. Amount_Within_Limit is neither a known fact nor the conclusion of any rule, so the system asks the user: **"Is the loan amount within the limit?"** The answer is No (Amount_Exceeds_Limit is true), so the goal fails. To obtain a useful outcome the system would then try the goal Refer_To_Manager through R6.

3.5 Conflict-resolution strategies

- **Rule order / priority:** the first applicable rule (or the one with the highest priority) fires. Simple and predictable, but depends on how the rules were ordered.
- **Specificity:** among applicable rules prefer the one with the most conditions, so exceptions override general rules.
- **Recency (and refraction):** prefer rules using the newest facts; refraction prevents the same rule firing twice on the same data.

3.6 Contradictory, redundant and circular rules

- **Contradictory** rules (one concludes Approve_Loan, another its opposite on the same facts) make the result depend on rule order and cannot both be right; they are found by consistency checks over pairs of rules with overlapping conditions.
- **Redundant** rules (duplicate or subsumed) waste time and complicate maintenance; **circular** rules (A → B and B → A) can loop in chaining or make a backward search never end. Both are found by dependency-graph checks and by testing with sample cases.

Analysis inquiry – forward vs backward chaining

Forward chaining is data-driven and preferable when many facts arrive and every consequence is wanted, as in monitoring, configuration or Case B's full derivation. **Backward chaining** is goal-driven and preferable when there is a specific hypothesis to confirm, as in diagnosis or Case A's question "can this loan be approved?", because it asks only the questions that matter and avoids deriving irrelevant facts.

Q4 – Semantic Networks and Frames (Library Domain) — 5 Marks

Problem Context – University Library. Consider the following description.

- Library members are either student members or faculty members. Every member has a name and membership ID, and by default may borrow 3 books for 14 days.
- Faculty members are an exception: they may borrow 10 books for 30 days.
- Books are either textbooks or reference books. Textbooks can be lent; reference books cannot be lent.
- Anil is a student member and Dr. Rao is a faculty member. 'Introduction to AI' is a textbook and 'Britannica' is a reference book. Anil has borrowed 'Introduction to AI'.

Your Challenge

1. Draw a semantic network showing is-a links, instance-of links and at least one other relation.
2. Represent the same knowledge as frames with slots, default values and inherited values.
3. Use inheritance to answer: How many books can Anil borrow? What is Dr. Rao's loan period? Can Anil borrow 'Britannica'?
4. Explain how a frame overrides a default value to handle an exception.
5. Compare semantic networks and frames and state one limitation of each.

Analysis Inquiry
What problem arises if a person is both a student member and a faculty member? Suggest how it can be resolved.

Evidence Expected: *Semantic network, frame hierarchy, inheritance reasoning, and comparison of the two schemes.*

Answer

4.1 Semantic network

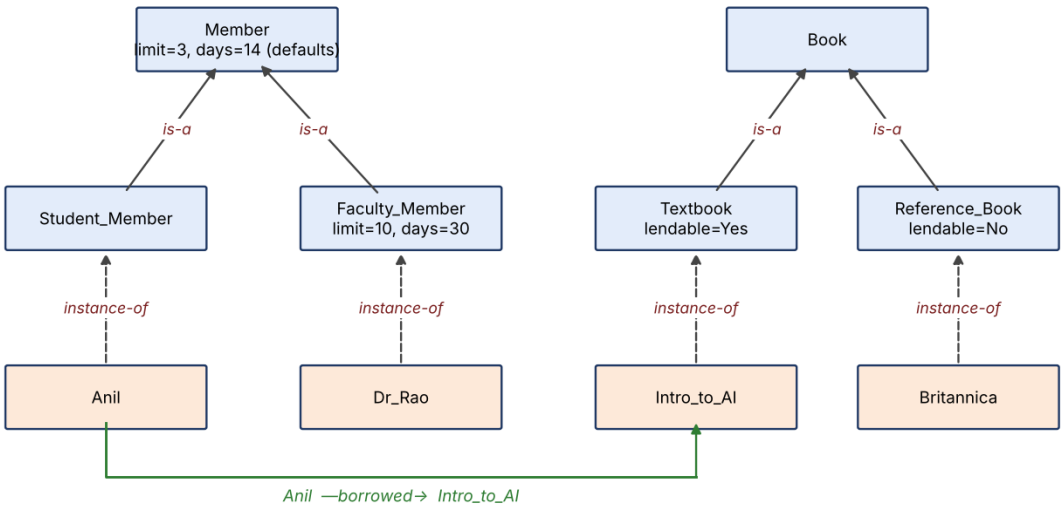


Figure 1 – Semantic network of the library domain

4.2 Frames

Frame	is-a / instance-of	Slots (default or own value)
Member	–	name, membership_id; borrow_limit = 3 (default); loan_days = 14 (default)
Student_Member	is-a Member	(inherits all slots)
Faculty_Member	is-a Member	borrow_limit = 10; loan_days = 30
Book	–	title; lendable
Textbook	is-a Book	lendable = Yes
Reference_Book	is-a Book	lendable = No
Anil	instance-of Student_Member	name = Anil; borrowed = Introduction_to_AI
Dr_Rao	instance-of Faculty_Member	name = Dr. Rao
Britannica	instance-of Reference_Book	title = Britannica

4.3 Answering queries by inheritance

- **How many books can Anil borrow?** Anil has no borrow_limit; look up Student_Member (none) then Member: **3** (default).
- **Dr. Rao's loan period?** Dr_Rao → Faculty_Member has loan_days = 30, found before Member's default: **30 days**.
- **Can Anil borrow 'Britannica'?** Britannica → Reference_Book has lendable = No, so **No**, regardless of Anil's limit.

4.4 Overriding defaults

A slot value is searched from the instance upward and the first value found is used. Faculty_Member defines its own borrow_limit and loan_days, which shadow the defaults in Member, so faculty are handled as an exception without changing the general rule or the other frames.

4.5 Comparison

Aspect	Semantic network	Frames
Structure	Nodes and labelled links; good for showing relationships	Objects with named slots, defaults, facets and procedures
Strength	Easy to read; inheritance along is-a links	Rich structure, defaults and exceptions; procedural attachment
Limitation	No standard meaning for links; weak for exceptions and n-ary relations	Becomes complex; inheritance conflicts; no formal semantics

Analysis inquiry – a student who is also faculty

The person inherits conflicting values from two parents (borrow_limit 3 and 10), the multiple-inheritance conflict. It can be resolved by a class precedence order, by a most-specific or most-permissive policy, or by giving the instance its own local slot value.

Q5 – Design an Expert System (Laptop Fault Diagnosis) — 7 Marks

Problem Context – Laptop Fault Diagnosis. A service centre wants an expert system that helps a front-desk technician diagnose common laptop faults (power, overheating, slow performance) from symptoms reported by the customer.

Your Challenge

1. Draw the architecture of the expert system and state the role of each component in this application.
2. Write at least six production rules for the knowledge base, with a certainty factor (CF) for each.
3. Show a sample consultation: facts supplied, rules fired and the final recommendation.
4. Show the explanation the system would give for a WHY question and for a HOW question.
5. Show how certainty factors are combined when two rules support the same conclusion, with the calculation.
6. Compare an expert system with a conventional program and state one limitation of expert systems.

> **Analysis Inquiry**
> Why is the separation of the knowledge base from the inference engine important?

Evidence Expected: *Architecture diagram, rule base, consultation trace, explanation facility examples, and certainty-factor calculation.*

Answer

5.1 Architecture

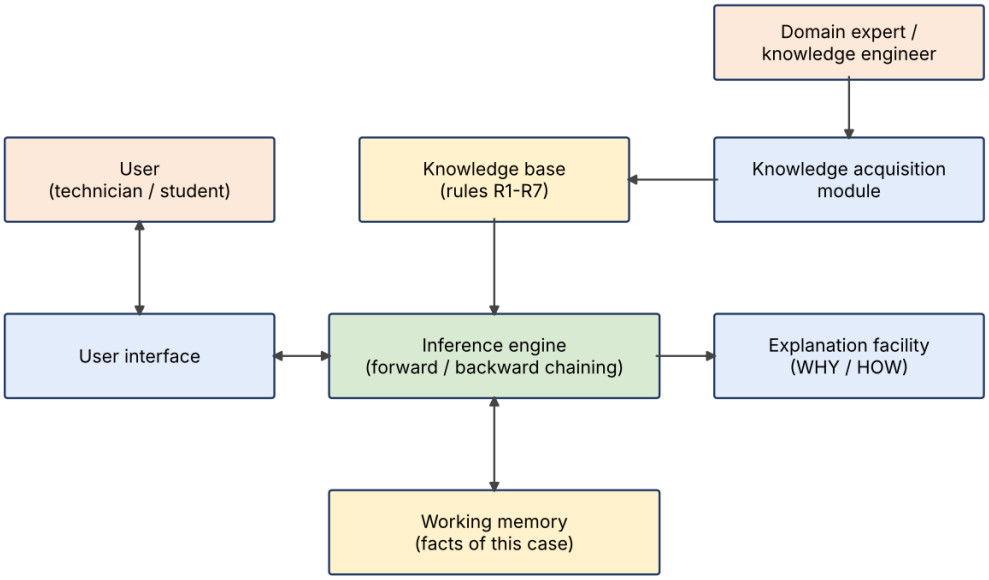


Figure 2 – Architecture of the laptop-diagnosis expert system

Component	Role in this system
Knowledge base	Rules R1–R7 about power, overheating and performance faults.
Working memory	Facts about the current laptop (symptoms and answers) and conclusions drawn so far.
Inference engine	Chooses and fires rules using forward or backward chaining and combines certainty factors.
User interface	Asks the technician questions and shows the diagnosis.
Explanation facility	Answers WHY a question is asked and HOW a conclusion was reached.
Knowledge acquisition module	Lets the expert / knowledge engineer add or change rules.

5.2 Knowledge base

Rule	IF	THEN	CF
R1	Not_Powering_On AND Charger_LED_Off	Faulty_Charger	0.8
R2	Not_Powering_On AND Charger_LED_On AND Battery_Drains_Fast	Faulty_Battery	0.7
R3	Overheating AND Fan_Noise_High	Dust_In_Fan	0.9
R4	Overheating AND Fan_Not_Spinning	Faulty_Fan	0.85
R5	Slow_Performance AND Overheating	Dust_In_Fan	0.5
R6	Slow_Performance AND High_Disk_Usage	Disk_Or_Software_Issue	0.6
R7	Dust_In_Fan	Recommend_Clean_Fan	1.0

5.3 Sample consultation

Customer reports: Overheating (certainty 0.9), Fan_Noise_High (0.8), Slow_Performance (0.7); Fan_Not_Spinning and High_Disk_Usage are answered No.

Step	Rule	Result
1	R3	Dust_In_Fan, CF = 0.9 × min(0.9, 0.8) = 0.72
2	R5	Dust_In_Fan, CF = 0.5 × min(0.7, 0.9) = 0.35
3	–	Combined CF(Dust_In_Fan) = 0.72 + 0.35 × (1 − 0.72) = 0.818
4	R7	Recommend_Clean_Fan, CF = 1.0 × 0.818 = 0.818

R4 and R6 do not fire because their conditions were answered No. Recommendation: clean the fan and heat-sink (certainty 0.82).

5.4 Explanations

- **WHY** (when the system asks “Is the fan noise high?”): “I am asking because rule R3 can conclude Dust_In_Fan if the laptop is overheating and the fan noise is high; overheating is already known.”
- **HOW** (for Recommend_Clean_Fan): “It was concluded by R7 from Dust_In_Fan (CF 0.818). Dust_In_Fan was supported by R3 (0.72, from Overheating and Fan_Noise_High) and R5 (0.35, from Slow_Performance and Overheating), combined into 0.818.”

5.5 Combining certainty factors

Each rule gives CF = rule CF × the minimum CF of its conditions. When two rules support the same conclusion with positive CFs, they are combined as CF = $CF_1 + CF_2 \times (1 - CF_1)$, which gives 0.818 above: two independent pieces of evidence raise confidence above either alone but never above 1.

5.6 Expert system vs conventional program

Aspect	Expert system	Conventional program
Knowledge	Explicit rules in a knowledge base, separate from control	Embedded in code
Problem type	Heuristic, uncertain, symbolic reasoning	Algorithmic, numeric processing
Explanation	Can explain WHY / HOW	Normally cannot
Change	Edit rules without reprogramming	Change and recompile code

Limitation: an expert system is brittle at the edge of its knowledge and has no common sense; it fails on faults not covered by its rules, and its rules are costly to acquire and keep up to date.

> **Analysis inquiry – separating the knowledge base from the inference engine**
> Separation lets experts update knowledge without touching the control logic, lets one engine be reused with other knowledge bases, makes the knowledge readable and testable, and makes explanation possible because the reasoning steps are rules that can be displayed.

Q6 – Ontologies and Knowledge Graphs (University Domain) — 5 Marks

Problem Context – University Knowledge Graph. A university wants to link information about students, professors, courses and departments so that questions can be answered across its separate databases.

Your Challenge

1. Design a small ontology with at least five classes, a class hierarchy, and at least four properties with domain and range.
2. Write at least eight facts as subject–predicate–object triples.
3. Draw the resulting knowledge graph.
4. Write and execute a graph query: “Which students are enrolled in courses taught by a named professor?”
5. Show one inference that follows from the class hierarchy or from a property’s domain or range.
6. Compare a taxonomy, an ontology and a knowledge graph.

> **Analysis Inquiry**
> How do ontologies support knowledge sharing and reuse? State one limitation.

Evidence Expected: *Ontology definition, triples, graph diagram, query trace, inference example and comparison.*

Answer

6.1 Ontology

Element	Definition
Classes	Person, Student, Professor, Course, Department
Hierarchy	Student ⊑ Person; Professor ⊑ Person
teaches	domain = Professor, range = Course
enrolledIn	domain = Student, range = Course
worksIn	domain = Professor, range = Department
offeredBy	domain = Course, range = Department

6.2 Triples (instances)

Subject	Predicate	Object
Dr_Meena	type	Professor
Dr_Meena	worksIn	CSE

Subject	Predicate	Object
Dr_Meena	teaches	ML101
ML101	offeredBy	CSE
Dr_Kumar	type	Professor
Dr_Kumar	worksIn	CSE
Dr_Kumar	teaches	DB102
DB102	offeredBy	CSE
Arun	type	Student
Arun	enrolledIn	ML101
Priya	type	Student
Priya	enrolledIn	DB102

6.3 Knowledge graph

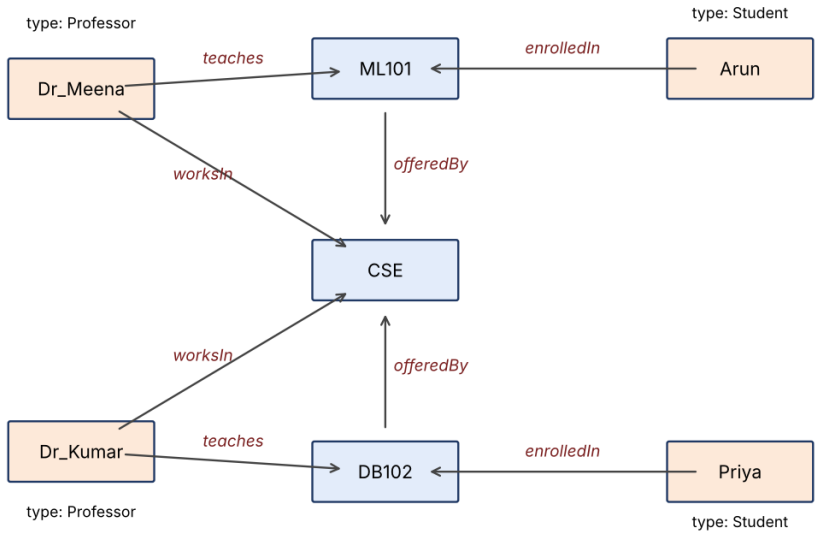


Figure 3 – Knowledge graph built from the triples

6.4 Graph query

Query: **{ ?s enrolledIn ?c . Dr_Meena teaches ?c }**. The pattern Dr_Meena teaches ?c matches only (Dr_Meena, teaches, ML101), so ?c = ML101. The pattern ?s enrolledIn ML101 matches (Arun, enrolledIn, ML101), so ?s = Arun. **Answer: Arun.** Priya is enrolled in DB102, which Dr_Meena does not teach.

6.5 Inference

- **Subclass:** Arun type Student and Student ⊑ Person, therefore Arun type Person, even though this triple was never stored.
- **Domain:** the domain of teaches is Professor, so any resource that teaches a course is inferred to be a Professor.

6.6 Taxonomy, ontology and knowledge graph

	Taxonomy	Ontology	Knowledge graph
Content	Hierarchy of terms (is-a only)	Classes, properties, constraints and axioms	Instances and their relations at scale
Reasoning	Inheritance only	Logical inference	Query and, with an ontology, inference
Example	Student ⊑ Person	teaches has domain Professor	Dr_Meena teaches ML101

Analysis inquiry – sharing, reuse and a limitation

An ontology gives a shared, formal vocabulary, so separate systems that use its terms interpret the data the same way. This allows different databases to be integrated, knowledge to be reused across applications, and implicit facts to be inferred. **Limitation:** building and maintaining an ontology is costly, and communities often disagree on how to model a concept, which limits agreement and adoption.

Q7 – Diagnose and Improve a Knowledge Management System — 13 Marks

Problem Context – IT Help-Desk Knowledge Base. A college keeps solutions to common IT problems (Wi-Fi, passwords, VPN, printing, software, exam portal) as articles in a searchable knowledge base. Students type a question and the system returns the best-matching article. After a semester, many students complain that the system returns no answer, an irrelevant answer or an outdated article. The faculty suspects limited or duplicated knowledge, an outdated or conflicting article, a flat and unstructured representation, poor search, or missing knowledge that only the support staff carry in their heads.

A. Diagnose the Knowledge Problem — 2 Marks

- Identify possible causes of the poor answers.
- Distinguish knowledge-quality problems (content) from system-design limitations (representation and retrieval).

Evidence Expected: *A diagnosis supported by example queries and the answers returned.*

B. Define the Knowledge Objective — 2 Marks

- Define the knowledge to be managed, its users and its representation.
- Specify a measurable objective for improving answer quality.

Evidence Expected: *A clearly defined task and a measurable objective.*

C. Select and Redesign the Method — 3 Marks

- Choose an approach, for example keyword search, rule-based troubleshooting, ontology/concept tagging, or knowledge graph.
- Justify the choice against the data and users, and propose a modification that addresses the limitation.

Evidence Expected: *Method selection, justification, and a revised knowledge-management workflow or diagram.*

D. Predict the Outcome — 1 Mark

- Predict how the change will affect the answers returned before you test it.

Evidence Expected: *A clear, testable prediction.*

E. Implement and Evaluate — 3 Marks

- Implement the original and revised methods using Python or a clearly traceable manual procedure.
- Evaluate both on the same set of test queries, report the results and identify remaining errors.

Evidence Expected: *Code or traceable calculations, a comparison table and evaluation results.*

F. Justify the Final Design — 2 Marks

- Decide whether the objective was achieved and explain the limitations of the final system.
- State when another approach would be preferable and how the knowledge will be kept up to date.

Evidence Expected: *An evidence-based conclusion and a justified recommendation.*

Answer

Scenario. Students' questions are matched by keywords to help-desk articles. The traceable dataset below is used for the whole investigation.

ID	Article (searchable words)	Concept tags	Status
A1	reset email password portal login	Account_Access	current
A2	connect campus wifi network ssid	Network	current
A3	install software lab license	Software	current
A4	vpn setup remote access old (2019)	Network, Remote_Access	retired
A5	vpn client remote access off-campus (2025)	Network, Remote_Access	current
A6	printer print queue lab	Printing	current
A7	id card request student office	Admin_Services	current
A8	exam portal login error	Account_Access, Exam_Portal	current

A. Diagnose the knowledge problem (2 marks)

Knowledge-quality problems (content)

- Outdated and duplicate knowledge: A4 (2019) and A5 (2025) both describe VPN setup; the old one is still searchable.
- Vocabulary gaps: students say "internet", "sign in" and "eduroam", but the articles say "wifi", "login" and "ssid".
- Tacit knowledge missing: support staff know that "eduroam" is the campus Wi-Fi, but this was never written down.

System-design limitations (representation and retrieval)

- Flat representation: articles are bags of words with no concepts, synonyms or status, so the system cannot tell that "internet" and "wifi" mean the same thing.
- Exact keyword matching with ties broken by order, so an outdated article can win.

Evidence: "internet not working in hostel", "unable to sign in to my account" and "eduroam keeps dropping" return nothing; "how to use vpn from home" returns the retired article A4 because A4 and A5 tie and A4 comes first. The main cause is the flat keyword representation combined with outdated and unlinked knowledge.

B. Define the knowledge objective (2 marks)

- **Knowledge to manage:** solutions to common IT problems; **users:** students; **task:** return the single correct current article for a free-text query.
- **Measurable objective:** raise top-1 accuracy on the 10 test queries from the keyword baseline to **at least 80 %**, return **no retired article**, and leave **at most 10 %** of queries unanswered.

C. Select and redesign the method (3 marks)

- **Choice:** lightweight ontology – concept tags with a synonym map – plus article lifecycle status, replacing flat keyword search. A full knowledge graph or expert system is unnecessary for 8 articles with a single question type.
- **Justification:** most failures are vocabulary mismatches and outdated content, which concept tagging and status control address directly; the approach is cheap to maintain, and the synonym map makes the staff's tacit vocabulary explicit.
- **Modification:** (1) retire A4; (2) map words to concepts ("internet", "wifi" → Network); (3) score articles by matched concepts, then by specificity (matches ÷ tags of the article).

Revised knowledge-management workflow:

```
Capture  (staff and student questions, unanswered queries)
   -> Validate (check accuracy, retire outdated or duplicate articles)
   -> Organise (concept tags, synonym map)
   -> Share    (concept-based retrieval for students)
   -> Maintain (review dates, log unanswered queries, add new terms)  --> back to Capture
```

D. Predict the outcome (1 mark)

Before running the test: retiring A4 and using the synonym map will fix the queries that failed from vocabulary mismatch or an outdated article, so top-1 accuracy should rise from about 60 % to **at least 80 %**, with no retired article returned. Queries using terms that are not in the synonym map, such as "eduroam", should still fail.

E. Implement and evaluate (3 marks)

Both methods are run on the same 10 queries with the code below.

```
# Help-desk knowledge base: keyword baseline vs concept-tagged retrieval
ARTICLES = [  # id, title words (searchable text), concept tags, status
    ("A1", "reset email password portal login",      ["Account_Access"],                "current"),
    ("A2", "connect campus wifi network ssid",       ["Network"],                       "current"),
    ("A3", "install software lab license",          ["Software"],                      "current"),
    ("A4", "vpn setup remote access old",            ["Network", "Remote_Access"],      "retired"),
    ("A5", "vpn client remote access off-campus",    ["Network", "Remote_Access"],      "current"),
    ("A6", "printer print queue lab",                ["Printing"],                      "current"),
    ("A7", "id card request student office",         ["Admin_Services"],                "current"),
    ("A8", "exam portal login error",                ["Account_Access", "Exam_Portal"], "current"),
]
QUERIES = [
    ("forgot my password", "A1"), ("internet not working in hostel", "A2"),
    ("wifi not working", "A2"), ("how to use vpn from home", "A5"),
    ("login to exam portal error", "A8"), ("unable to sign in to my account", "A1"),
    ("print my assignment in lab", "A6"), ("install matlab on laptop", "A3"),
    ("lost student card", "A7"), ("eduroam keeps dropping", "A2"),
]
SYNONYMS = {}
for words, concept in [("password forgot reset credentials sign login account", "Account_Access"),
                       ("wifi internet network", "Network"),
                       ("vpn remote home", "Remote_Access"),
                       ("exam portal", "Exam_Portal"),
                       ("print printer printing", "Printing"),
                       ("install software matlab licence", "Software"),
                       ("card id", "Admin_Services")]:
    for w in words.split():
        SYNONYMS[w] = concept


def keyword_search(query):                      # original method
    best, best_score = None, 0
    for aid, text, _, _ in ARTICLES:
        score = sum(1 for w in query.split() if w in text.split())
        if score > best_score:                  # ties keep the earlier article
            best, best_score = aid, score
    return best


def concept_search(query):                      # revised method
    wanted = {SYNONYMS[w] for w in query.split() if w in SYNONYMS}
    best, best_key = None, (0, 0)
    for aid, _, tags, status in ARTICLES:
        if status != "current":                 # retired knowledge is never returned
            continue
        matched = len(wanted & set(tags))
        key = (matched, matched / len(tags))    # more matches, then more specific
        if matched > 0 and key > best_key:
            best, best_key = aid, key
    return best
```

```
status = {a[0]: a[3] for a in ARTICLES}
for name, method in [("Keyword baseline", keyword_search), ("Concept-tagged", concept_search)]:
    results = [(q, exp, method(q)) for q, exp in QUERIES]
    correct = sum(got == exp for _, exp, got in results)
    none = sum(got is None for _, _, got in results)
    outdated = sum(got is not None and status[got] == "retired" for _, _, got in results)
    print(f"{name}: correct {correct}/10 = {correct*10}%, unanswered {none}, "
          f"outdated returned {outdated}")
    for q, exp, got in results:
        if got != exp:
            print("   wrong:", q, "-> expected", exp, "got", got)
```

Output of the program:

```
Keyword baseline: correct 6/10 = 60%, unanswered 3, outdated returned 1
   wrong: internet not working in hostel -> expected A2 got None
   wrong: how to use vpn from home -> expected A5 got A4
   wrong: unable to sign in to my account -> expected A1 got None
   wrong: eduroam keeps dropping -> expected A2 got None
Concept-tagged: correct 9/10 = 90%, unanswered 1, outdated returned 0
   wrong: eduroam keeps dropping -> expected A2 got None
```

#	Query	Expected	Keyword baseline	Concept-tagged
1	forgot my password	A1	A1	A1
2	internet not working in hostel	A2	none	A2
3	wifi not working	A2	A2	A2
4	how to use vpn from home	A5	A4 (retired)	A5
5	login to exam portal error	A8	A8	A8
6	unable to sign in to my account	A1	none	A1
7	print my assignment in lab	A6	A6	A6
8	install matlab on laptop	A3	A3	A3
9	lost student card	A7	A7	A7
10	eduroam keeps dropping	A2	none	none

Method	Top-1 accuracy	Unanswered	Outdated article returned	Remaining errors
Keyword baseline	6/10 = 60 %	3	1	Queries 2, 4, 6, 10
Concept-tagged (revised)	9/10 = 90 %	1	0	Query 10 (“eduroam” not in the synonym map)

F. Justify the final design (2 marks)

- **Objective achieved?** Yes on this evidence: accuracy rose from 60 % to 90 % (target ≥ 80 %), no retired article was returned, and 10 % were unanswered (target ≤ 10 %). The prediction in part D was confirmed, including the expected failure on “eduroam”.
- **Limitations:** only 8 articles and 10 queries; the synonym map was built while looking at the same queries, so 90 % is optimistic and must be re-checked on new, held-out queries; the synonym map needs manual upkeep; scoring ignores word order and negation.
- **Keeping the knowledge current:** assign an owner and review date to each article, log unanswered queries, and add terms such as “eduroam” to the synonym map. This turns the staff’s tacit knowledge into explicit knowledge.
- **When another method is preferable:** with hundreds of articles or free-form questions, use a statistical or embedding-based search engine, or a full knowledge graph with reasoning; for step-

by-step fault finding, a rule-based troubleshooting system that asks questions is better than article retrieval.

Mark Distribution

Question	Assessed Area	Marks
Q1	Knowledge representation	6
Q2	Knowledge acquisition	6
Q3	Production rules and inference (chaining)	8
Q4	Semantic networks and frames	5
Q5	Expert systems	7
Q6	Ontologies and knowledge graphs	5
Q7	Diagnose, redesign and evaluate a knowledge management system	13
Total		50